AETHER

Test Strategy (48-hour prototype)

What gets tested in the vertical slice, and what deliberately does not.

Project Autonomous Margin & Collateral Intelligence **Programme** GIFT IFIH Young Builders **Revision** B
Generated 2026-08-15

> **PROTOTYPE DOCUMENT**
> Describes a prototype under construction. All market data, counterparties, portfolios and legal documents referenced are **simulated** and clearly labelled as such in the repository. Nothing here asserts regulatory approval, certification under any industry methodology, or connectivity to any market infrastructure.

Scope: what gets tested in the vertical slice, and — as importantly — what deliberately does not. This is a 48-hour solo build, so the strategy is chosen for defect-catching value per minute spent, not for coverage optics.

The one-line rationale

> Test the arithmetic exhaustively. Test the plumbing once. Test the demo end to end.

Everything below follows from that. The Risk Engine and Optimizer are where a wrong answer is both *likely* and *invisible* — a plausible-looking margin number is far more dangerous than a stack trace. That is where the effort goes.

Layer 1 — Golden reference tests `@golden`

The primary defense. Hand-computed CSA cases whose expected values are derived independently of the implementation, committed as fixtures.

Discipline that makes them worth anything: **the expected value must be computed by hand before the function is written.** A "golden" test written by running the code and pasting the output tests nothing except that the code is deterministic.

CASE	ASSERTS
Exposure below Threshold	No call generated. The Threshold gate works.
Call above Threshold, below MTA	No transfer. The MTA gate works.
Call above both	Delivery Amount produced and correctly rounded up to USD 100,000
Return Amount	Rounds **down**, opposite direction to delivery
MTA tested before rounding	Per CSA 3(b). An amount of 480,000 is gated out even though it would round to 500,000
Ratings Event applied	Party B Threshold drops to zero, call increases accordingly
Negative exposure	`max(0, ...)` floors correctly; no negative Credit Support Amount
FX haircut	INR asset valued at 94% × 92% = 86.48%, matching the CSA illustration

The last row is a gift from the fixture: CSA Paragraph 11(g)(i) contains a worked illustration (`94% × 92% = 86.48%`), so the document itself supplies an independently authored expected value.

Layer 2 — Property-based invariants `@property` (Hypothesis)

Generated inputs, asserting things that must hold for *every* input. These catch the cases no one thinks to write down.

Risk Engine - Credit Support Amount is never negative. - Increasing the Threshold never increases the Delivery Amount (monotonicity). - A delivery rounded up never leaves coverage below the requirement. - Return Amount never exceeds the value of the Credit Support Balance. - Trades in different netting sets never offset.

Optimizer - Any returned allocation is **feasible**: post-haircut value ≥ required amount. - No asset is allocated beyond its available quantity. - No ineligible asset ever appears in an allocation. - **`optimizer_cost ≤ baseline_cost`, always.** If the LP ever loses to the cash-first heuristic, either the objective or the baseline is wrong. This single invariant is the most valuable test in the suite, because it protects the product's central claim. - Concentration limits hold in every returned allocation.

Layer 3 — Determinism `@determinism`

Run the complete pipeline twice from identical inputs; assert **byte-identical audit chain hashes**. This test *is* the reproducibility claim — without it, "reproducible" is marketing.

Also asserts: no `datetime.now()` reachable from `core/`, enforced by a grep-style import guard test over the package.

Layer 4 — Extraction eval harness

Scores the Legal-Clause Agent against `fixtures/ground_truth/csa_meridian_2026.expected.json`. Per-field:

```
field_score = 0.5 · value_match + 0.3 · citation_match + 0.2 · review_flag_match
```

Targets: value accuracy ≥ 0.90, citation accuracy ≥ 0.80, **review-flag recall = 1.00**.

Review-flag recall is the one target set at 100%, and failures there are weighted 3×. Missing the ratings-conditional Threshold means confidently computing a margin call on a Threshold that may legally be zero — a wrong number presented as certain. That is the single worst failure mode in the system, so it is the one the eval refuses to tolerate.

The harness runs in `replay` mode against cached extractions, so it works with no network.

Layer 5 — Wrong-way risk guard

A dedicated test, called out separately because the fixture contains a deliberate trap: `US0000C2Y09` is paper issued by **Meridian Global Markets plc — the counterparty itself**. It is cheap (3bp) and eligible on its face under CSA item (H), so a naive optimizer will reach for it first.

CSA Paragraph 11(g)(iv) prohibits it. **If this asset appears in any allocation, the eligibility filter is broken.** Asserted directly.

Layer 6 — Vertical slice smoke test

One test that runs the whole pipeline and asserts it completes with a decision and a verifiable audit chain:

```
CSA fixture → extraction (replay) → risk calculation → optimization → decision → audit
```

This is the demo path. If it goes red, the demo is broken, and that must be visible in CI rather than on stage.

Explicitly not tested in the 48-hour build

Each of these is a deliberate call, not an oversight.

NOT TESTED	WHY
Frontend component unit tests	The slice-level test covers the path that matters. Component tests would be churn against a UI still being designed.
API contract snapshots	Only one consumer, written by the same person, in the same repo. The type generator catches drift more cheaply.
Live-mode LLM calls	Non-deterministic and network-dependent. The eval harness runs against cached extractions instead.
Load / performance	Demo scale is a handful of trades and ten assets. The LP solves in milliseconds.
Database migrations	SQLite, recreated from fixtures. There is no production data to migrate.
Auth / authorization	Not built. See the architecture document's out-of-scope list.
Browser E2E (Playwright)	Deferred. Valuable, but it is second-order until the slice runs. Reconsider once the UI stabilises.

Coverage posture

`core/` should approach total coverage — it is pure, so there is no excuse. Everything else is covered incidentally by the slice test. **No coverage percentage is a gate.** Chasing a number would pull time away from the golden cases, which are worth far more.

Commands

```
python scripts/verify_env.py            # environment gate — run first
pytest backend/tests -q                 # everything
pytest backend/tests -m golden -q       # reference cases
pytest backend/tests -m property -q     # invariants
```

Order of construction

Tests for the quantitative core are written **alongside** the functions, not after. For the golden cases specifically, the expected values are written **first** — a golden test authored after the fact is a regression test wearing a golden test's clothes, and it will happily enshrine a bug.